

Week 7: Operational Semantics & Hoare Logic

1 Operational Semantics

This section is about the small-step operational semantics of While programs as given by the relation $\rightarrow \subseteq \mathcal{C} \times \mathcal{C}$ where the configurations are either a statement and a state or a state or a state $\mathcal{C} = (\mathcal{S} \times \text{State}) \cup \text{State}$, which is defined by these following rules:

$$\begin{array}{c}
 \frac{}{\langle \text{skip}, \sigma \rangle \rightarrow \sigma} \qquad \frac{}{\langle x \leftarrow e, \sigma \rangle \rightarrow \sigma[x \mapsto \llbracket e \rrbracket_{\mathcal{A}}(\sigma)]} \\
 \\
 \frac{\langle S_1, \sigma_1 \rangle \rightarrow \sigma_2}{\langle S_1; S_2, \sigma_1 \rangle \rightarrow \langle S_2, \sigma_2 \rangle} \qquad \frac{\langle S_1, \sigma_1 \rangle \rightarrow \langle S'_1, \sigma_2 \rangle}{\langle S_1; S_2, \sigma_1 \rangle \rightarrow \langle S'_1; S_2, \sigma_2 \rangle} \\
 \\
 \frac{}{\langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \rightarrow \langle S_1, \sigma \rangle} \llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \top \\
 \\
 \frac{}{\langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \rightarrow \langle S_2, \sigma \rangle} \llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \perp \\
 \\
 \frac{}{\langle \text{while } e \text{ do } S, \sigma \rangle \rightarrow \sigma} \llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \perp \\
 \\
 \frac{}{\langle \text{while } e \text{ do } S, \sigma \rangle \rightarrow \langle S; \text{while } e \text{ do } S, \sigma \rangle} \llbracket e \rrbracket_{\mathcal{B}}(\sigma) = \top
 \end{array}$$

Figure 1: Operational semantics for While.

- * 1. Calculate the terminal state and write down a trace for the statement $x \leftarrow 1; \{x \leftarrow 2; x \leftarrow 3\}$ with the initial state $[]$ using the rules in Figure 1. Remember, variables are assigned 0 by default.
- * 2. Calculate the terminal state and write down a trace for the statement $\{x \leftarrow 1; x \leftarrow x * 2\}; x \leftarrow x + y$ with the initial state $[x \mapsto 2, y \mapsto 2]$ using the rules in Figure 1.
- * 3. Find a state σ such that $\langle x \leftarrow 1; y \leftarrow x * 2, [] \rangle \rightarrow^* \sigma$. You should provide the corresponding trace.

* 4. Compute the final state for the program if $x \leq y$ then $x \leftarrow y$ else $y \leftarrow x$ when executed in each of the following states:

- $[]$
- $[x \mapsto 2, y \mapsto 3]$
- $[x \mapsto 4, y \mapsto 2]$

* 5. Find a state $\sigma \in \text{State}$ such that $\text{while } !(x \leq -1) \text{ do } x \leftarrow x + d, [d \mapsto -1] \rightarrow^* \sigma$. You should provide the corresponding trace.

* 6. Find a state $\sigma \in \text{State}$ such that $\langle x \leftarrow 2; y \leftarrow x * y, \sigma \rangle \rightarrow^* [x \mapsto 2, y \mapsto 4]$. You should provide the corresponding trace.

** 7. Suppose $e \in \mathcal{B}$ is a Boolean expression that is semantically equivalent to false. Prove that $\langle \text{while } e \text{ do } S, \sigma \rangle \rightarrow \sigma$ for any state $\sigma \in \text{State}$.

** 8. Suppose $S_1, S_2 \in \mathcal{S}$ are two statements such that $\langle S_1, \sigma \rangle \rightarrow^* \sigma'$ and $\langle S_2, \sigma \rangle \rightarrow^* \sigma'$ for some states $\sigma, \sigma' \in \text{State}$. Prove that $\langle \text{if } e \text{ then } S_1 \text{ else } S_2, \sigma \rangle \rightarrow^* \sigma'$ for any Boolean expression $e \in \mathcal{B}$.

** 9. Suppose we introduce a new language construct “do S while e ” where $S \in \mathcal{S}$ is a statement and $e \in \mathcal{B}$ is a Boolean expression. The operational semantics for this construct is given by the following inference rules:

$$\frac{}{\langle \text{do } S \text{ while } e, \sigma \rangle \rightarrow \langle S; \text{if } e \text{ then do } S \text{ while } e \text{ else skip}, \sigma \rangle}$$

- (a) Find a state $\sigma \in \text{State}$ such that $\langle \text{do } x \leftarrow x + 1 \text{ while } x \leq 1, [] \rangle \rightarrow^* \sigma$ and give the associated trace.
- (b) For a given statement $S \in \mathcal{S}$ and a Boolean expression $e \in \mathcal{B}$ find a While program that is equivalent to the program do S while e but does not use the new construct. That is, find a statement $S' \in \mathcal{S}$ such that:

$$\langle S', \sigma \rangle \rightarrow^* \sigma' \Leftrightarrow \langle \text{do } S \text{ while } e, \sigma \rangle \rightarrow^* \sigma'$$

You do not need to prove that your answer is correct but should provide a trace for $\langle S', [] \rangle \rightarrow^* \sigma$ where S is given to be the statement $x \leftarrow x + 1$, where e is given to be the expression $x \leq 1$, and where σ is the state from part (a).

** 10. Suppose we introduce a new language construct for x do S where $S \in \mathcal{S}$ is a statement and $x \in \text{Var}$ is a variable. The operational semantics for this construct is given by the following inference rules:

$$\frac{}{\langle \text{for } x \text{ do } S, \sigma \rangle \rightarrow \sigma} \sigma(x) \leq 0 \quad \frac{}{\langle \text{for } x \text{ do } S, \sigma_1 \rangle \rightarrow \langle S; \text{for } x \text{ do } S, \sigma[x \mapsto \sigma(x) - 1] \rangle} \sigma(x) > 0$$

- (a) Find a state $\sigma \in \text{State}$ such that $\langle \text{for } x \text{ do } y \leftarrow y + x; x \leftarrow x - 2, [x \mapsto 3] \rangle \rightarrow^* \sigma$ and give the associated trace.
- (b) For a given statement $S \in \mathcal{S}$ and a variable $x \in \text{Var}$ find a While program that is equivalent to the program for x do S but does not use the new construct. That is, find a statement $S' \in \mathcal{S}$ such that:

$$\langle S', \sigma \rangle \rightarrow^* \sigma' \Leftrightarrow \langle \text{for } x \text{ do } S, \sigma \rangle \rightarrow^* \sigma'$$

You do not need to prove that your answer is correct but should provide a trace for $\langle S', [x \mapsto 3] \rangle \rightarrow^* \sigma$ where S is given to be the statement $y \leftarrow y + x; x \leftarrow x - 2$ and σ is the state from part (a).

- *** 11. Show that, if $y \notin \text{FV}(e_1)$ and $x \notin \text{FV}(e_2)$, then the statements $x \leftarrow e_1; y \leftarrow e_2$ and $x \leftarrow e_1; y \leftarrow e_2$ equivalent in the sense that $\langle x \leftarrow e_1; y \leftarrow e_2, \sigma \rangle \rightarrow^* \sigma'$ if and only if $\langle y \leftarrow e_2; x \leftarrow e_1, \sigma \rangle \rightarrow^* \sigma'$. You may use results from the previous worksheet.

- *** 12. The set of variables *modified* by a statement is defined by recursion as follows:

$$\begin{aligned} \text{mod}(\text{skip}) &= \emptyset \\ \text{mod}(x \leftarrow e) &= \{x\} \\ \text{mod}(S_1; S_2) &= \text{mod}(S_1) \cup \text{mod}(S_2) \\ \text{mod}(\text{if } e \text{ then } S_1 \text{ else } S_2) &= \text{mod}(S_1) \cup \text{mod}(S_2) \\ \text{mod}(\text{while } e \text{ do } S) &= \text{mod}(S) \end{aligned}$$

- (a) Prove that if $\langle S_1, \sigma_1 \rangle \rightarrow \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \text{mod}(S_1)$.
- (b) Prove that if $\langle S_1, \sigma_1 \rangle \rightarrow \langle S_1, \sigma_2 \rangle$ then $\text{mod}(S_2) \subseteq \text{mod}(S_1)$ and $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \text{mod}(S_1)$. You may use the previous result.
- (c) Prove that if $\langle S_1, \sigma_1 \rangle \rightarrow^* \sigma_2$ then $\sigma_1(x) = \sigma_2(x)$ for all $x \notin \text{mod}(S_1)$. You may use the previous results, and the fact that $\gamma_1 \rightarrow^* \gamma_2$ if, and only if, $\gamma_1 \rightarrow^n \gamma_2$ for some $n \geq 0$.

Hoare Logic

- * 13. Consider the following *invalid* Hoare triple: $\{x \leq y\} y \leftarrow y * 2 \{x \leq y\}$. Find an initial state $\sigma \in \text{State}$ and a trace from the configuration $\langle y \leftarrow y * 2, \sigma \rangle$ that contradicts this triple.
- * 14. Find a statement S such that $\{\text{true}\} S \{z \leq x \ \&\& \ z \leq y\}$.
- ** 15. For each of the following statements, compute the strongest post-condition from the pre-condition $(\exists q. x = q * y) \ \&\& \ y \geq 0$:

(a) $x \leftarrow x * x$

(b) if $y \leq x$ then $x \leftarrow x - y$ else $y \leftarrow y - x$

** 16. Using your answers to the previous question, for each of the following Hoare triples determine whether they are valid or not. You should justify your answer.

(a) $\{(\exists q. x = q * y) \ \&\& \ y \geq 0\} \ x \leftarrow x * x \ \{(\exists q. x = q * y)\}$

(b) $\{(\exists q. x = q * y) \ \&\& \ y \geq 0\} \ x \leftarrow x * x \ \{x \geq y\}$

(c) $\{(\exists q. x = q * y) \ \&\& \ y \geq 0\} \text{ if } y \leq x \text{ then } x \leftarrow x - y \text{ else } y \leftarrow y - x \ \{(\exists q. x = q * y) \ \&\& \ y \geq 0\}$

** 17. Suppose $S_1, S_2, S_3 \in \mathcal{S}$ are statements satisfying the following Hoare triples:

- $\{x \leq y\} \ S_1 \ \{y \leq x \ \&\& \ x \leq 0\}$
- $\{\forall z. x * z = 0\} \ S_2 \ \{y = z\}$
- $\{x > y \ \&\& \ z = x\} \ S_3 \ \{y \leq x\}$

Then which of the following triples can be derived? You should briefly justify your answer.

(a) $\{x \leq y \ \&\& \ x = 0\} \ S_1; S_2 \ \{y = z\}$

(b) $\{z = x\} \text{ if } x \leq y \text{ then } S_1 \text{ else } S_2 \ \{y \leq x\}$

(c) $\{x \leq y\} \ S_1; \text{ if } y = 0 \text{ then } S_2 \text{ else } y \leftarrow z \ \{y = z\}$

*** 18. The rule of consequence allows us to weaken a Hoare triple for more general context. However, it is also useful to be able to adapt Hoare triples to contexts with other unrelated variables. This can be done through the *constancy rule*:

$$\frac{\{P\} \ S \ \{Q\}}{\{P \ \&\& \ R\} \ S \ \{Q \ \&\& \ R\}} \text{FV}(R) \cap \text{mod}(S) = \emptyset$$

Where $\text{FV}(P)$ for a predicate $P \subseteq \text{State}$ is defined as the set of variables $x \in \text{Var}$ such that $\sigma \in P$ but $\sigma[x \mapsto n] \notin P$ for some state $\sigma \in \text{State}$ and $n \in \mathbb{Z}$; intuitively, those variables whose value effect whether a state is in the set of not and $\text{mod}(S)$ is the set of variables appearing anywhere in the program.

- (a) Suppose that S_1 and S_2 are two statement such that $\{P_1\} \ S_1 \ \{Q_1\}$ and $\{P_2\} \ S_2 \ \{Q_2\}$ where:
- $\text{FV}(P_1), \text{FV}(Q_1) \subseteq \text{FV}(S_1)$;
 - $\text{FV}(P_2), \text{FV}(Q_2) \subseteq \text{FV}(S_2)$;
 - And $\text{FV}(S_1) \cap \text{FV}(S_2) = \emptyset$.

Justify how $\{P_1 \wedge P_2\} \ S_1; S_2 \ \{Q_1 \wedge Q_2\}$ can be derived from the constancy rule.

(b) Suppose that $P \subseteq \text{State}$ is some set of states. Prove that, if $\sigma \in P$ and $\sigma(x) = \sigma'(x)$ for all $x \in \text{FV}(P)$ and $\#\{x \in \text{Var} \mid \sigma(x) \neq \sigma'(x)\}$ is finite, then $\sigma' \in P$. You may wish to prove it by induction on $\#\{x \in \text{Var} \mid \sigma(x) \neq \sigma'(x)\}$.

(c) Prove that the rule of constancy holds that is:

$$\{P\} S \{Q\} \Rightarrow \{P \ \&\& \ R\} S \{Q \ \&\& \ R\}$$

for any statement S , and predicates $P, Q, R \subseteq \text{State}$ such that $\text{FV}(R) \cap \text{mod}(S) = \emptyset$. You may wish to use the result from Question 12.